



La Programmation Orientée Objet en C++

Il y a plusieurs façons d'utiliser le C++. On peut être utilisateur de classes ou concepteur de classes. Dans les deux cas, définir, concevoir et implémenter des classes sont les activités primaires des développeurs C++.

Par où on commence ? En général, on démarre avec une abstraction ou un concept. On considère un truc à coder et puis on essaie de voir comment on va gérer son ou ses fonctions, ce qui est interne et ce qui est public et comment on va l'utiliser... Au démarrage de cette réflexion, il faut se mettre la casquette de celui qui va utiliser la classe. Notre exemple de concept pour cet article sera un élément graphique à dessiner comme une ligne, un rectangle, une ellipse, etc. En anglais, c'est un shape.

```
// C'est une déclaration de classe
class CShape;

// C'est la future manière avec laquelle je veux dessiner des Shapes
bool Draw(const CShape& item);
```

Ensuite on envisage la classe dans son ensemble :

```
class CShape
{
public:
    // interface publique

private:
    // implémentation privée
};
```

Qu'est-ce que l'on met en private/public et comment va-t-elle être structurée. On commence petit bras et puis la classe prend du volume.

Constructeur et Destructeur

La première question qui vient à l'esprit est comment vais-je créer mon objet et ai-je besoin de lui fournir des paramètres ? Avec ma classe CShape, on peut se dire, je n'en ai pas besoin mais aussi j'en ai besoin. Les deux possibilités existent ! Soit c'est une figure connue -> d'où une énumération pour les types connus, et, autre possibilité qui consiste à dessiner une image, et là, il me faut un nom de fichier ; tout est ouvert !

```
enum ShapeType
{
    line,
    circle,
    rectangle,
    left_arrow,
    right_arrow,
    picture
};
```

L'avantage de l'énumération est que c'est évalué à la compilation. Il ne peut pas y avoir d'erreur sur le nommage car c'est un type explicite contrairement à une simple chaîne de caractères. Voyons sommairement comment pourrait être construite cette fameuse classe CShape...

```
class CShape
{
public:
    CShape();
    CShape(int x, int y, int xx, int yy, ShapeType type);
    CShape(int x, int y, int xx, int yy, std::string fileName);
    virtual ~CShape();

private:
    std::string m_fileName;
    ShapeType m_type;
};
```

J'y vois deux façons de créer un shape. Soit à partir de l'énumération soit à partir d'un fichier pour les images. Pour le moment, je ne sais pas comment organiser le dessin du shape. Dois-je le mettre à l'intérieur de la classe ou à l'extérieur ? Je ne sais pas... Il faut du temps pour considérer quelle sera la façon la plus naturelle de réaliser cette opération. Dans le monde Windows GDI+, pour dessiner un élément il faut un handle particulier qui possède toutes les primitives de dessins. En pensant comment allait être dessiné un élément, on peut envisager d'inclure une méthode Draw dans la classe CShape. La première pensée, c'est de se dire, je vais implémenter tous les cas possibles dans Draw avec une construction qui ressemble à ça :

```
void Draw(const CDrawingContext& ctx) const
{
    if (m_type == ShapeType::line)
    {
        ctx.Line(m_rect.left, m_rect.bottom,
            m_rect.top, m_rect.right);
    }
    if (m_type == ShapeType::rectangle)
    {
        ctx.Rectangle(m_rect);
    }
    // TODO
}
```

Dans la classe, il est important de marquer les méthodes qui ne modifient pas les données private. Pour faire cela, on leur ajoute le

mot-clé `const` à la fin comme c'est indiqué sur la méthode `Draw`. De plus, lorsque l'on passe un argument qui n'a pas vocation à être modifié, on le passe en référence `const`. Comme on peut le voir ci-dessus, la définition d'une classe est un travail itératif dans lequel les meilleures idées chassent celles d'avant ou les conforte. Si une classe est bien conçue, la lecture de ses membres public est limpide car on va à l'essentiel. Attention, les membres private expliquent aussi beaucoup de choses sur les détails d'implémentation. Maintenant, mettons-nous dans un contexte où il existe plusieurs classes et les principes vus ci-dessus ne suffisent pas à faire un modèle objet. Il nous faut des mécanismes plus sophistiqués pour relier les classes les unes avec les autres.

Les classes possèdent des associations et des comportements, et c'est ici que les Jedi se positionnent pour y concevoir un modèle élégant, facile à utiliser et qui rend les services voulus.

Notre méthode `Draw` n'est pas OOP car si je rajoute un type dans l'énumération, je dois ajouter un `if()` dans la méthode `Draw()` et coder l'implémentation. On peut faire mieux, beaucoup mieux.

Les concepts de OOP

Les deux piliers de l'OOP sont l'héritage et le polymorphisme. L'héritage permet de grouper les classes en familles de types et permet de partager des opérations et des comportements et des données. Le polymorphisme permet de déclencher des opérations dans ces familles à un niveau unitaire. Ainsi ajouter ou supprimer une classe n'est pas trop un problème.

L'héritage définit une relation parent/enfant. Le parent définit l'interface publique et l'implémentation private est commune à tous ses enfants. Chaque enfant choisit s'il veut hériter d'un comportement unique ou bien le surcharger. En C++, le parent est appelé « classe de base » et l'enfant « classe dérivée ». Le parent et les enfants définissent une hiérarchie de classes. Le parent est souvent une classe abstraite et les enfants implémentent les méthodes virtuelles pures pour que l'ensemble fonctionne.

Dans un programme OOP, on manipule les classes via un pointeur sur la classe de base plutôt que sur les objets dérivés.

Une classe abstraite

Revenons sur la classe `CShape`... La prochaine étape dans le design est de créer une classe abstraite pour identifier les opérations propres à chaque item – ce qui implique des opérations avec une implémentation basée sur une classe dérivée. Ces opérations sont des fonctions virtuelles pures de la classe de base. Une méthode virtuelle pure est une méthode abstraite. Il n'y a pas de corps. Cela implique que la classe devient abstraite aussi. Il n'est pas possible de créer un objet à partir d'une classe abstraite. La méthode `Draw()` doit être définie comme virtuelle pure car il y a plusieurs types de dessin à faire et que chaque dessin est propre à une classe donnée. Le corps de `Draw()` n'existe pas, c'est la raison pour laquelle on dit que la classe est abstraite ; il y a au moins une méthode virtuelle pure. Voici donc comment on fait :

```
class CShapeEx
{
public:
    CShapeEx() {}
    CShapeEx(ShapeType type, RECT rect, const std::string &fileName)
```

```
: m_type(type), m_rect(rect), m_fileName(fileName) {}
    virtual ~CShapeEx() {}

public:
    virtual void Draw(const CDrawingContext& ctx) const = 0;
    virtual void DrawTracker(const CDrawingContext& ctx) const = 0;

private:
    std::string m_fileName;
    ShapeType m_type;
    RECT m_rect;
};
```

Et nous allons maintenant déclarer des classes dérivées de `CShapeEx` :

```
class CRectangle : public CShapeEx
{
public:
    CRectangle() {}
    virtual ~CRectangle() {}

public:
    virtual void Draw(const CDrawingContext& ctx) const
    {
        // TODO
        std::cout << "Rectangle::Draw" << std::endl;
    }

    virtual void DrawTracker(const CDrawingContext& ctx) const
    {
        // TODO
    }
};

class CLine : public CShapeEx
{
public:
    CLine() {}
    virtual ~CLine() {}

public:
    virtual void Draw(const CDrawingContext& ctx) const
    {
        // TODO
        std::cout << "Line ::Draw" << std::endl;
    }

    virtual void DrawTracker(const CDrawingContext& ctx) const
    {
        // TODO
    }
};
```

Si on veut rajouter un item à dessiner dans la hiérarchie, il suffit de rajouter une classe dérivée ! Maintenant nous allons voir comment fonctionne ce mécanisme de `virtual`...

```

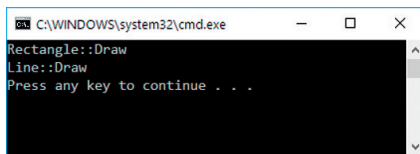
CDrawingContext ctxt;
CRectangle * pRect = new CRectangle();
CLine * pLine = new CLine();
CShapeEx * ptr = nullptr;

ptr = pRect;
ptr->Draw(ctxt);

ptr = pLine;
ptr->Draw(ctxt);

```

On commence par déclarer 2 items à dessiner. Puis le manager, un pointeur sur la classe de base CShapeEx est déclaré. A chaque fois qu'il pointe sur un objet d'une classe dérivée, la fonction virtuelle Draw() appelée est celle de la classe dérivée car cette fonction est définie comme virtuelle.



Maintenant, il nous manque quelque chose de très utile qui évitera les usines à gaz de construction d'objets dérivés. Il nous faut une factory ! Un mécanisme de création d'objet.

La factory

```

class CFactory
{
private:
    CFactory();

public:
    static CShapeEx * CreateObject(ShapeType type)
    {
        if (type == ShapeType::line)
        {
            CLine * pObj = new CLine();
            pObj->m_type = type;
            return pObj;
        }

        if (type == ShapeType::rectangle)
        {
            CRectangle * pObj = new CRectangle();
            pObj->m_type = type;
            return pObj;
        }

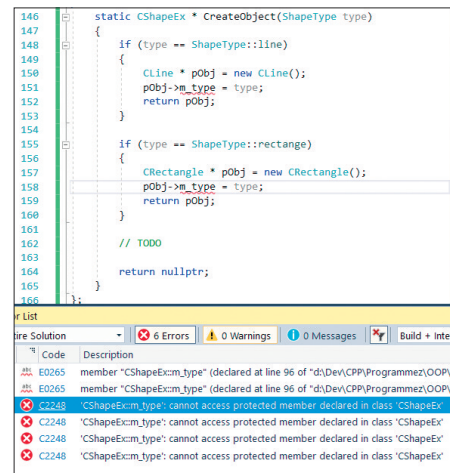
        // TODO

        return nullptr;
    }
};

```

Cette classe permet via une méthode static de créer un objet en fonction de son type dans l'énumération. Le type est affecté en variable membre de l'objet créé. On notera que le constructeur est private, cela veut dire que l'on ne peut pas déclarer d'objet CFacto-

ry. On ne peut qu'utiliser la méthode static CreateObject. Sauf que, la compilation tombe en erreur :



Le membre m_type est inaccessible vu son niveau de protection... On va donc ajouter quelque chose à la classe CShapeEx :

```

class CShapeEx
{
public:
    CShapeEx() {}
    CShapeEx(ShapeType type, RECT rect, const std::string &fileName)
        : m_type(type), m_rect(rect), m_fileName(fileName) {}
    virtual ~CShapeEx() {}

public:
    virtual void Draw(const CDrawingContext& ctxt) const = 0;
    virtual void DrawTracker(const CDrawingContext& ctxt) const = 0;

private:
    std::string m_fileName;
    ShapeType m_type;
    RECT m_rect;

    // La classe CFactory peut avoir accès aux membres...
    friend class CFactory;
};

```

On ajoute en fin de classe que la classe CFactory est amie de la classe CShapeEx. Et là, il n'y a plus d'erreur de compilation ! Les puristes comme AlainZ, à Redmond, vous diront que cela viole la mécanique objet et que c'est une construction exotique et qu'il vaut mieux passer par le constructeur pour passer le type... Je ne suis pas de cet avis. Je pense que c'est élégant de faire une entorse pour la factory. Voici maintenant le code qui appelle la factory :

```

CDrawingContext ctxt;
CShapeEx * pRect = CFactory::CreateObject(ShapeType::rectangle);
CShapeEx * pLine = CFactory::CreateObject(ShapeType::line);
CShapeEx * ptr = nullptr;

ptr = pRect;
ptr->Draw(ctxt);

ptr = pLine;
ptr->Draw(ctxt);

```

Et le résultat est équivalent.

Compilation sous Linux

Pour ceux qui ne le savent pas, le langage C++ est normalisé ISO. Sur la plateforme Linux, là où tout est fait en C/C++, le langage possède plusieurs compilateurs dont le célèbre GCC. Vous pouvez utiliser Visual Studio Code que fournit Microsoft comme IDE pour compiler sous Linux via le terminal.

Il faut installer l'extension C++. Moi j'ai installé Cygwin et MSYS. Ainsi je dispose de commandes Linux. **1**

Une touche de C++ 11

On va essayer de continuer notre application de dessin en matérialisant la zone de dessin. Cela ressemble à un ensemble de shapes sur un support de dessin. Première étape quand on pense au concept de la planche à dessin c'est quoi ? Le stockage des éléments graphiques dans un container STL. Oui Monsieur, quand on veut stocker quelque chose, on passe par les containers STL. On va donc utiliser le `std::vector<T>` de la STL. On va stocker des shapes un peu particulières que sont les shapes partagées. Partagées ? Par qui ? Dans un vrai logiciel de dessin, on affiche les propriétés dans une grille et le dessin est sur le panneau central, donc on partage nos objets et ce n'est pas que de l'affichage. Je vous montre le résultat. **2**

Sur cet écran, on voit que le rectangle gris sélectionné a ses attributs directement dans la grille de propriétés mais aussi sur le Ribbon. Comment faire pour partager les données entre le panneau de dessin, le Ribbon et la grille de propriétés ? Est-ce que l'on fait des copies des objets ? Oui, mais on fait des copies de pointeurs et non des copies d'objets. Pour faire cela, on utilise le mécanisme des pointeurs partagés de la STL. Voyons son utilisation. Premièrement, on va concevoir graphiquement la zone de dessin.

La zone de dessin

Cette zone contient les objets shape à dessiner. On va donc créer un `vector<shared_ptr<CShapeEx>>`. Le container vector est défini dans `<vector>`. On va avoir une collection d'éléments partagés `CShapeEx`. Ainsi, on pourra stocker ce pointeur particulier où on veut sans se soucier de la libération de la mémoire. La zone de dessin est une vue fille MDI en jargon Windows. Elle possède deux scrollbars, une verticale et une horizontale. Si on veut faire une gestion de `shared_ptr<T>`, il faut faire évoluer la factory :

```
static std::shared_ptr<CShapeEx> CreateObject(ShapeType type)
```

```
{
    std::shared_ptr<CShapeEx> pObj = nullptr;
```

```
    if (type == ShapeType::line)
```

```
    {
        pObj = std::make_shared<CLine>();
        pObj->m_type = type;
        return pObj;
    }
```

```
    if (type == ShapeType::rectangle)
```

```
    {
        pObj = std::make_shared<CRectangle>();
        pObj->m_type = type;
        return pObj;
    }
```

```
}

return nullptr;

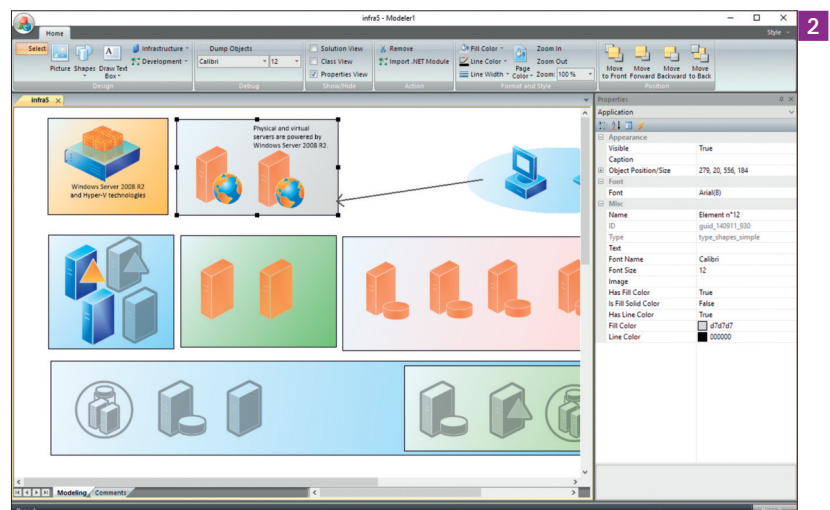
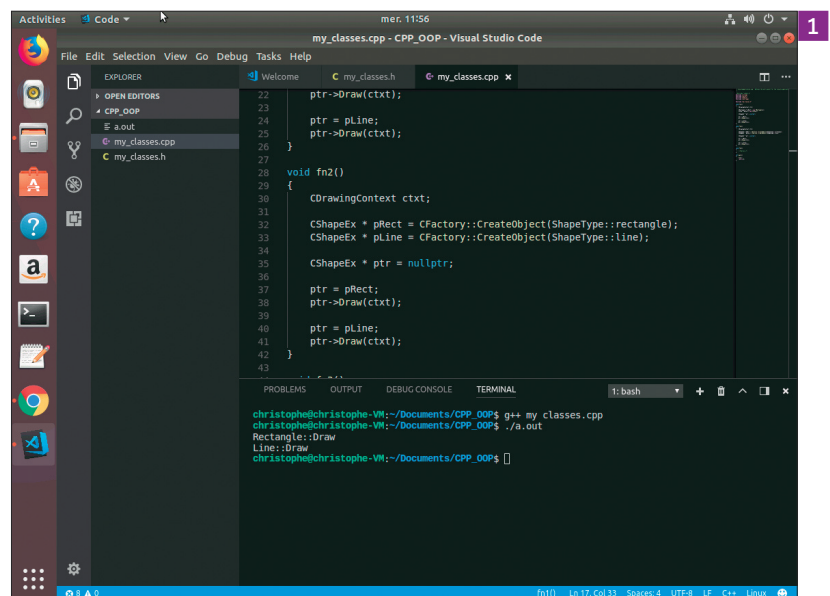
}
```

On ne fait plus un `new()` mais un appel à `std::make_shared<T>()`. De plus, dans l'utilisation des objets, cela donne ça :

```
CDrawingContext ctxt;
std::shared_ptr<CShapeEx> pRect = CFactory::CreateObject(ShapeType::rectangle);
std::shared_ptr<CShapeEx> pLine = CFactory::CreateObject(ShapeType::line);
std::shared_ptr<CShapeEx> ptr = nullptr;
ptr = pRect;
ptr->Draw(ctxt);
ptr = pLine;
ptr->Draw(ctxt);
```

Il existe même une façon de laisser le compilateur déterminer le type de variable utilisée en utilisant `auto` :

```
CDrawingContext ctxt;
auto pRect = CFactory::CreateObject(ShapeType::rectangle);
```



```
auto pLine = CFactory::CreateObject(ShapeType::line);
auto ptr = pRect;
ptr->Draw(ctx);
ptr = pLine;
ptr->Draw(ctx);
```

Revenons à notre design de classes. Le container de shapes est juste une collection `std::vector<T>` de `std::shared_ptr<CShapeEx>` :

```
class CElementContainer : public CObject
{
private:

public:
    DECLARE_SERIAL(CElementContainer);
    CElementContainer();
    virtual ~CElementContainer(void);

    // ...
    // overridden for document i/o
    virtual void Serialize(CElementManager * pElementManager, CArchive & ar);
    // Debugging Operations
public:
    void DebugDumpObjects(CModeler1View * pView);

    // Operations MFC like
public:
    std::shared_ptr<CElement> GetHead();
    int GetCount();
    void RemoveAll();
    void Remove(std::shared_ptr<CElement> pElement);
```

```
3 // Managing UI dependencies (Ribbon UI, Property Grid, ClassView)
public:
    void UpdateUI(CModeler1View * pView, std::shared_ptr<CElement> pElement);
    void UpdateRibbonUI(CModeler1View * pView, std::shared_ptr<CElement> pElement);
    void UpdatePropertyGrid(CModeler1View * pView, std::shared_ptr<CElement> pElement);
    void UpdateClassView();
    void UpdateClassView(std::shared_ptr<CElement> pElement);
    void UpdateFileView();
    void UpdateFileView(std::shared_ptr<CElement> pElement);

    // Managing Object Format
public:
    void FillColor(CModeler1View * pView);
    void NoFillColor(CModeler1View * pView);
    void LineColor(CModeler1View * pView);
    void LineWidth(CModeler1View * pView, UINT nID);
    void PageColor(CModeler1View * pView);

    // Managing Zoom Operations
public:
    void Zoom(CModeler1View * pView);
    void ZoomIn(CModeler1View * pView);
    void ZoomOut(CModeler1View * pView);

    // Load Module Operations
public:
    void LoadModule(CModeler1View * pView);

    // Managing Object Selection
public:
    bool HasSelection();
    bool IsSelected(std::shared_ptr<CElement> pElement);
    bool Select(std::shared_ptr<CElement> pElement);
    bool Deselect(std::shared_ptr<CElement> pElement);
    void SelectNone();
    void DrawSelectionRect(CModeler1View * pView);

    // Overridables
public:
    virtual void PrepareDC(CModeler1View * pView, CDC * pDC, CPrintInfo * pInfo);
    virtual void Draw(CModeler1View * pView, CDC * pDC);
    virtual void DrawEx(CModeler1View * pView, CDC * pDC);
    virtual void Update(CModeler1View * pView, LPARAM lHint, CObject * pHint);

    // UI Handlers
public:
    virtual void OnLButtonDown(CModeler1View * pView, UINT nFlags, const CPoint & cpoint);
    virtual void OnLButtonDblClick(CModeler1View * pView, UINT nFlags, const CPoint & cpoint);
    virtual void OnLButtonUp(CModeler1View * pView, UINT nFlags, const CPoint & cpoint);
    virtual void OnMouseMove(CModeler1View * pView, UINT nFlags, const CPoint & cpoint);
};
```

```
void AddTail(std::shared_ptr<CElement> pElement);
```

```
// Attributes
private:
    vector<std::shared_ptr<CElement>> m_objects;
};
```

Là, je vous montre un vrai code et les éléments sont matérialisés par la classe `CElement` mais c'est pareil que `CShapeEx`. On notera que les fonctions de gestion au vector sont des méthodes helpers dans cette classe. On notera aussi que la fonction `Serialize` qui permet de faire Load/Save est gérée dans cette classe. En effet, si il existe des données à mettre dans un fichier de données, c'est bien cette classe. Il faut maintenant intégrer cette classe dans la mécanique de fenêtrage Windows. Dans un canevas MFC, on possède un `Document` pour y stocker les données et une `View` pour y faire les opérations d'affichage.

Abstraction

La couche de dessin est confiée à une classe explicite et les méthodes ne sont pas directement insérées dans la vue. Pourquoi un tel niveau de poupées russes ? Parce que sinon, c'est crado. C'est trop entrelacé dans les couches Windows et le code est spaghetti. J'ai donc une classe qui se nomme `CElementManager` qui fait le lien avec la `View` MFC. De ce fait beaucoup de méthodes prennent en paramètre une `View`... C'est de la délégation de comportement. Mais dans cette classe on trouve tous les éléments de la zone de dessin :

- Les objets à afficher ;
- Les éléments d'une sélection ;
- Les éléments du presse-papier ;
- La couleur de fond de la zone ;
- Le mode du pointeur de souris ;
- Le facteur de zoom ;
- Un booléen pour savoir si on est en train de dessiner un gabarit ;
- L'élément courant avec son type et son id et son nom.

Voici un aperçu du fichier d'entêtes de la classe `CElementManager` : 3

Cette classe est juste dépendante d'une `View` sur laquelle il va y avoir des opérations et des manipulations d'affichage. Exemple de méthode :

```
void CElementManager::ZoomIn(CModeler1View * pView)
{
    m_fZoomFactor += 0.10f;
    Invalidate(pView);
}
```

Conclusion

Pour un code orienté objet, il ne faut pas hésiter à faire des constructions, créer des classes et leur donner un minimum de responsabilités. De toute façon, si vous créez une classe et que vous n'y trouvez aucune méthode à mettre dedans, vous devrez faire machine arrière... La construction orientée objet est importante dans le développement moderne et on retrouvera les mêmes principes dans les autres langages comme Java ou C#.

Le code de l'application graphique est disponible ici :

<https://github.com/ChristophePichaud/UltraFluidModeler>